COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CpE21S3
DATE PERFORMED:	10/6/2026	**DATE SUBMITTED:**	10/6/2026
NAME:	Wayne Kaisen C. Paraiso	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A:
A1:

mainA1.cpp > ...

```cpp
#include <iostream>
#include <vector>

struct Edge {
    int from, to, weight;
};

class Graph {
    std::vector<int> vertices;   // vertex labels
    std::vector<Edge> edges;     // edge list

    bool hasVertex(int v) const {
        for (int x : vertices)
            if (x == v) return true;
        return false;
    }

public:
    // Add a vertex
    void addVertex(int v) {
        if (hasVertex(v)) {
            std::cerr << "Vertex " << v << " already exists\n";
            return;
        }
        vertices.push_back(v);
    }

```

```
    // Add an edge from u to v with a weight
    void addEdge(int u, int v, int weight = 1) {
        if (!hasVertex(u) || !hasVertex(v)) {
            std::cerr << "Vertex out of bounds: (" << u << ", " << v << ")\n";
            return;
        }
        edges.push_back({u, v, weight});
    }

    // Display the vertices
    void displayVertices() const {
        std::cout << "Vertices: ";
        for (int v : vertices) std::cout << v << " ";
        std::cout << "\n";
    }

    // Display the edges
    void displayEdges() const {
        std::cout << "Edges (from -> to : weight):\n";
        for (const Edge &e : edges)
            std::cout << "  " << e.from << " -> " << e.to << " : " << e.weight << "\n";
    }
};

int main() {
    Graph g;

    for (int i = 0; i < 6; i++) g.addVertex(i);

    g.addEdge(0, 1, 2);
    g.addEdge(0, 2, 4);
    g.addEdge(1, 4, 3);
    g.addEdge(2, 3, 2);
    g.addEdge(3, 1, 4);
    g.addEdge(4, 3, 3);

    g.displayVertices();
    std::cout << "\n";
    g.displayEdges();
    return 0;
}
```

```
d:\CPE010\HOA 10.1>cd "d:\CPE010\HOA
Vertices: 0 1 2 3 4 5

Edges (from -> to : weight):
  0 -> 1 : 2
  0 -> 2 : 4
  1 -> 4 : 3
  2 -> 3 : 2
  3 -> 1 : 4
  4 -> 3 : 3
```

A2:

mainA2.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <iomanip>

// Graph using an adjacency matrix
class MatrixGraph {
    int V;                                  // number of vertices
    std::vector<std::vector<int>> matrix;   // 0 means no edge

public:
    MatrixGraph() : V(0) {}

    // add a vertex
    void addVertex() {
        for (auto &row : matrix) row.push_back(0);   // new column
        matrix.push_back(std::vector<int>(V + 1, 0)); // new row
        V++;
    }

    // add an edge from u to v
    void addEdge(int u, int v, int weight = 1) {
        if (u < 0 || u >= V || v < 0 || v >= V) {
            std::cerr << "Vertex out of bounds: (" << u << ", " << v << ")\n";
            return;
        }
        matrix[u][v] = weight;
    }

```

```
    // show the vertices
    void displayVertices() const {
        std::cout << "Vertices: ";
        for (int i = 0; i < V; i++) std::cout << i << " ";
        std::cout << "\n";
    }

    // show the edges
    void displayEdges() const {
        std::cout << "Edges (from -> to : weight):\n";
        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                if (matrix[i][j] != 0)   // skip empty spots
                    std::cout << "  " << i << " -> " << j << " : " << matrix[i][j] << "\n";
    }

    // show the matrix
    void displayMatrix() const {
        std::cout << "Adjacency matrix:\n    ";
        for (int j = 0; j < V; j++) std::cout << std::setw(3) << j;
        std::cout << "\n";
        for (int i = 0; i < V; i++) {
            std::cout << std::setw(3) << i << " ";
            for (int j = 0; j < V; j++) std::cout << std::setw(3) << matrix[i][j];
            std::cout << "\n";
        }
    }
};

int main() {
    MatrixGraph g;

    // add 6 vertices (0 to 5)
    for (int i = 0; i < 6; i++) g.addVertex();

    // add edges (from, to, weight)
    g.addEdge(0, 1, 2);
    g.addEdge(0, 2, 4);
    g.addEdge(1, 4, 3);
    g.addEdge(2, 3, 2);
    g.addEdge(3, 1, 4);
    g.addEdge(4, 3, 3);

    g.displayVertices();
    std::cout << "\n";
    g.displayEdges();
    std::cout << "\n";
    g.displayMatrix();
```

```
    return 0;
}
```

```
Adjacency matrix:
      0  1  2  3  4  5
  0   0  2  4  0  0  0
  1   0  0  0  0  3  0
  2   0  0  0  2  0  0
  3   0  4  0  0  0  0
  4   0  0  0  3  0  0
  5   0  0  0  0  0  0
```

A3:

```cpp
#include <iostream>
#include <vector>

// Graph using an adjacency list
class ListGraph {
    struct Neighbor { int to, weight; };

    int V;                                          // number of vertices
    std::vector<std::vector<Neighbor>> list;    // neighbors of each vertex

public:
    ListGraph() : V(0) {}

    // add a vertex
    void addVertex() {
        list.push_back({});   // empty list for the new vertex
        V++;
    }

    // add an edge from u to v
    void addEdge(int u, int v, int weight = 1) {
        if (u < 0 || u >= V || v < 0 || v >= V) {
            std::cerr << "Vertex out of bounds: (" << u << ", " << v << ")\n";
            return;
        }
        list[u].push_back({v, weight});
    }

    // show the vertices
    void displayVertices() const {
```

```
        std::cout << "Vertices: ";
        for (int i = 0; i < V; i++) std::cout << i << " ";
        std::cout << "\n";
    }

    // show the edges
    void displayEdges() const {
        std::cout << "Edges (from -> to : weight):\n";
        for (int u = 0; u < V; u++)
            for (const auto &n : list[u])
                std::cout << "  " << u << " -> " << n.to << " : " << n.weight << "\n";
    }

    // show the adjacency list
    void displayList() const {
        std::cout << "Adjacency list (vertex -> neighbor(weight)):\n";
        for (int i = 0; i < V; i++) {
            std::cout << "  " << i << " ->";
            if (list[i].empty()) std::cout << " (none)";   // no edges
            for (const auto &n : list[i])
                std::cout << " " << n.to << "(" << n.weight << ")";
            std::cout << "\n";
        }
    }
};
```

```
int main() {
    ListGraph g;

    // add 6 vertices (0 to 5)
    for (int i = 0; i < 6; i++) g.addVertex();

    // add edges (from, to, weight)
    g.addEdge(0, 1, 2);
    g.addEdge(0, 2, 4);
    g.addEdge(1, 4, 3);
    g.addEdge(2, 3, 2);
    g.addEdge(3, 1, 4);
    g.addEdge(4, 3, 3);

    g.displayVertices();
    std::cout << "\n";
    g.displayEdges();
    std::cout << "\n";
    g.displayList();
    return 0;
}
```

```
Adjacency list (vertex -> neighbor(weight)):
  0 -> 1(2) 2(4)
  1 -> 4(3)
  2 -> 3(2)
  3 -> 1(4)
  4 -> 3(3)
  5 -> (none)
```

In ILO A, I implemented the same weighted directed graph (6 vertices, 6 edges) in three representations. The edge list in A1 stores a vector with from, to and weight. It is the easiest to build, but hasVertex() has to scan the whole vertex vector, and finding the edges of one vertex means scanning every edge. The adjacency matrix in A2 uses a vector> where 0 means "no edge". addVertex() has to add a new column to every row and a new row to the matrix, and the matrix needs $O(V^2)$ space no matter how many edges exist. Its advantage is that looking up a single edge is O(1). It is also not possible to store an edge of weight 0 in the matrix, since 0 is already used as "no edge". The printed matrix is not symmetric (for example, 0→1 is 2 but 1→0 is 0), which confirms the graph is directed, and row 5 is all zeros because vertex 5 has no outgoing edges. The adjacency list in A3 stores a vector per vertex. It takes up O(V+E) space and allows me to go straight to a vertex's neighbors, and vertex 5 correctly shows (none). All three representations produced the same vertices and edges, so it is a matter of trade-offs. The matrix is good for dense graphs and fast edge lookups, and the list is good for sparse graphs and traversals.

ILO B:
B1:

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

// an edge in the graph
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // compare edges by weight only
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```
29
30  // print the graph
31  template <typename T>
32  std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
33  {
34      for (auto i = 1; i < G.vertices(); i++)
35      {
36          os << i << ":\t";
37          auto edges = G.outgoing_edges(i);
38          for (auto &e : edges)
39              os << "{" << e.dest << ": " << e.weight << "}, ";
40          os << std::endl;
41      }
42      return os;
43  }
44
45  // graph stored as an edge list
46  template <typename T>
47  class Graph
48  {
49  public:
50      // create a graph with N vertices
51      Graph(size_t N) : V(N)
52      {
53      }
54
```

```
55      // number of vertices
56      auto vertices() const
57      {
58          return V;
59      }
60
61      // all edges
62      auto &edges() const
63      {
64          return edge_list;
65      }
66
67      // add an edge (vertex IDs start at 1)
68      void add_edge(Edge<T> &&e)
69      {
70          if (e.src >= 1 && e.src <= V &&
71              e.dest >= 1 && e.dest <= V)
72              edge_list.emplace_back(e);
73          else
74              std::cerr << "Vertex out of bounds" << std::endl;
75      }
76
77      // edges going out of vertex v
78      auto outgoing_edges(size_t v) const
```

```
79      {
80          std::vector<Edge<T>> edges_from_v;
81          for (auto &e : edge_list)
82          {
83              if (e.src == v)
84                  edges_from_v.emplace_back(e);
85          }
86          return edges_from_v;
87      }
88
89      // lets us use std::cout << graph
90      template <typename U>
91      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
92
93  private:
94      size_t V;
95      std::vector<Edge<T>> edge_list;
96  };
97
98  // depth-first search using a stack
99  template <typename T>
100 auto depth_first_search(const Graph<T> &G, size_t dest)
101 {
```

```
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1);   // start at vertex 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // only visit a vertex once
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // push neighbors that are not visited yet
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
```

```cpp
}

// build the sample graph
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

template <typename T>
```

```cpp
void test_DFS()
{
    // make the graph and print it
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // run DFS from vertex 1 and print the visit order
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

B2:

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

// an edge in the graph
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
```

```
27  };
28
29  // print the graph
30  template <typename T>
31  std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
32  {
33      for (auto i = 1; i < G.vertices(); i++)
34      {
35          os << i << ":\t";
36          auto edges = G.outgoing_edges(i);
37          for (auto &e : edges)
38              os << "{" << e.dest << ": " << e.weight << "}, ";
39          os << std::endl;
40      }
41      return os;   // the sheet leaves this out, it is needed
42  }
43
44  // graph stored as an edge list
45  template <typename T>
46  class Graph
47  {
48  public:
49      Graph(size_t N) : V(N) {}
50
```

```
51      auto vertices() const
54      }
55
56      auto &edges() const
57      {
58          return edge_list;
59      }
60
61      // add an edge (vertex IDs start at 1)
62      void add_edge(Edge<T> &&e)
63      {
64          if (e.src >= 1 && e.src <= V &&
65              e.dest >= 1 && e.dest <= V)
66              edge_list.emplace_back(e);
67          else
68              std::cerr << "Vertex out of bounds" << std::endl;
69      }
70
71      // edges going out of vertex v
72      auto outgoing_edges(size_t v) const
73      {
74          std::vector<Edge<T>> edges_from_v;
75          for (auto &e : edge_list)
76          {
77              if (e.src == v)
```

```
78                  {
79                      edges_from_v.emplace_back(e);
80                  }
81              }
82              return edges_from_v;
83          }
84
85          // U avoids reusing the name T
86          template <typename U>
87          friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
88
89      private:
90          size_t V;
91          std::vector<Edge<T>> edge_list;
92      };
93
94      // build the sample graph (weights are not needed for BFS)
95      template <typename T>
96      auto create_reference_graph()
97      {
98          Graph<T> G(9);
```

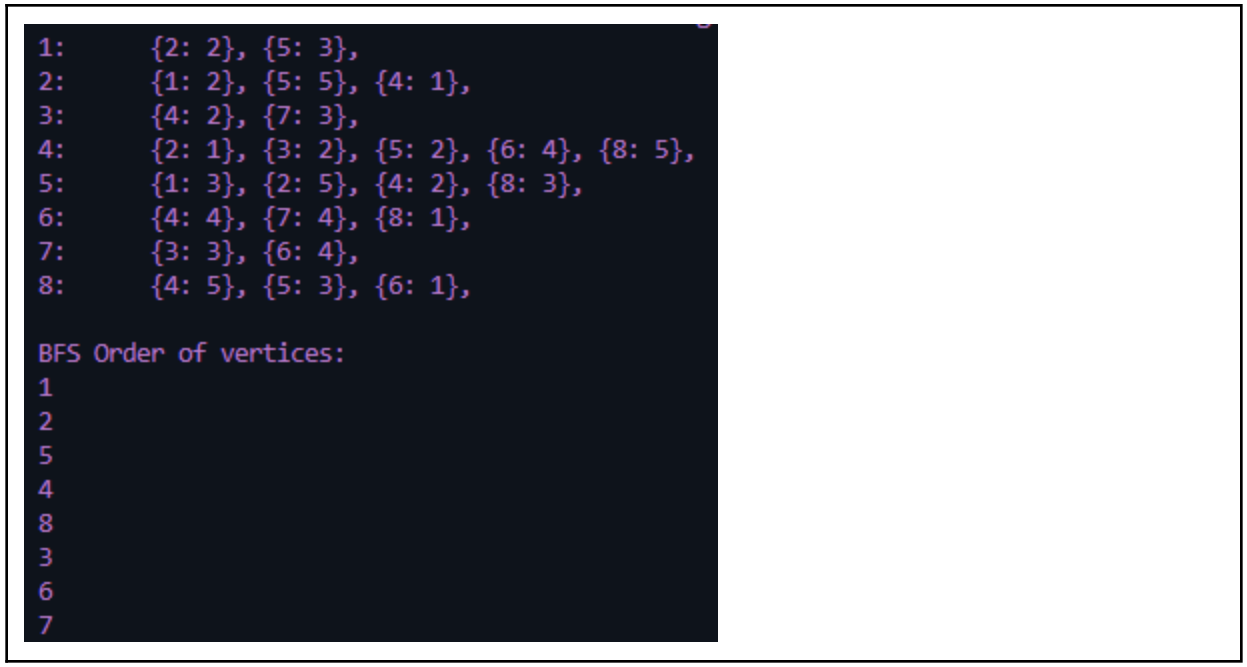
```
99
100     std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
101     edges[1] = {{2, 2}, {5, 3}};
102     edges[2] = {{1, 2}, {5, 5}, {4, 1}};
103     edges[3] = {{4, 2}, {7, 3}};
104     edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
105     edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
106     edges[6] = {{4, 4}, {7, 4}, {8, 1}};
107     edges[7] = {{3, 3}, {6, 4}};
108     edges[8] = {{4, 5}, {5, 3}, {6, 1}};
109
110     for (auto &i : edges)
111         for (auto &j : i.second)
112             G.add_edge(Edge<T>{i.first, j.first, j.second});
113
114     return G;
115 }
116
117 // breadth-first search using a queue
118 template <typename T>
119 auto breadth_first_search(const Graph<T> &G, size_t dest)
120 {
121     std::queue<size_t> queue;
```

```
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1);   // start at vertex 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // only visit a vertex once
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS()
{
    // make the graph and print it
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // run BFS from vertex 1 and print the visit order
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```



In ILO B, I implemented a templated Graph stored as an edge list, with 1-based vertex IDs and an operator<< to print it. For B1, I implemented DFS using a std::stack, a std::set to keep track of visited vertices, and a vector to store the visit order. Starting at vertex 1, I got 1, 5, 8, 6, 7, 3, 4, 2. The order is upside down because the last neighbor pushed on to the stack is the first one popped off, meaning that the search will go as far as possible along one branch (1→5→8→6→7→3→4) before backtracking to 2. For B2, I replaced the stack with a std::queue and got BFS order 1, 2, 5, 4, 8, 3, 6, 7. The traversal goes level by level, where the levels are 1; 2 and 5; 4 and 8; 3 and 6; and 7. The only structural difference from DFS was using top()/pop() instead of front()/pop(), and making sure that a vertex was not added to the queue more than once. One thing I noticed was that the outgoing_edges() method scans the entire edge list for every call, which makes both traversals O(V·E) instead of the expected O(V+E), which an adjacency list would have fixed. I also had to add the return os; that the sheet forgot to put in operator<<.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1.
The best algorithm for the given task is the Depth-First Search (DFS). The idea is quite simple: the person goes as far as possible from the starting location, following one route, and then backtracking to the first location and going in another direction. More precisely, once the person reaches a dead end (i.e., a vertex with no unvisited neighbors), they backtrack along the edge to the previous vertex and attempt to continue exploring. The process continues until all vertices have been visited. This traversal method utilizes a stack (or recursion) and a visited marker to keep track of the exploration progress; the former stores the nodes in the order they should be visited, while the latter keeps an account of nodes that have been processed already. DFS is not suitable for this task since it would require visiting all neighbors of a given node before proceeding further.

2.
The tree analogy of DFS is preorder traversal. It visits the node first and then each of its children from left to right in depth. Just like the graph DFS algorithms, preorder goes as far as it can from the starting node and then backtracks to the nearest ancestor node. The key difference is that the graph DFS requires a visited set because it has cycles, unlike trees.

```
  / | \
 2  7  8
/ \
3  6
```

Pseudocode:

```
preorder(node):
    if node is empty: return
    visit(node)
    for each child c: preorder(c)
```

Code:

```
void preorder(Node* n) {
    if (n == nullptr) return;
    std::cout << n->val << " ";
    for (Node* c : n->children) preorder(c);
}
```

3.
A queue (std::queue) is used for BFS. The vertices are pulled from the front (front() and pop()) and their neighbors are pushed to the back (push()), resulting in the level-by-level order. Additionally, a std::set is utilized to keep track of visited vertices, and a std::vector is used to store the visit order.

4.
A node is visited only once in BFS. In the lab code a vertex can be pushed into the queue more than once, because several neighbors may point to it. But when it comes out of the queue, the `visited` check skips it if it was already processed. Marking each vertex as visited is what keeps BFS from repeating work or looping forever on cycles.

Conclusion:
I think I did well on this activity. I was able to construct all three graph representations and the generated results matched the required outputs. Additionally, I was able to properly implement DFS and BFS algorithms and was able to explain how and why their visit order differs (1 5 8 6 7 3 4 2 and 1 2 5 4 8 3 6 7). I was also able to identify trade-offs between the edge list, adjacency matrix and adjacency list, and not only know how to code them but understand why. For improvement, I should learn more about templates, friend functions and operators overloading, as the implementation of graph was the hardest part to read for me. I should use more efficient structures, such as adjacency list within the templated graph so that outgoing_edges() does not have to traverse the whole list of edges. Finally, I would like to improve my recursive depth-first search, test disconnected graphs and implement additional features such as handling invalid vertex indices or adding duplicate edges.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction, Code Assistance, Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Have a draft of the conclusion and code analysis, asked claude to fix code errors I cant find, asked claude to explain certain parts I do not understand**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.